

Oprogramowanie Systemowe

Linux – procesy

mgr inż. Marek Downar



Proces

- Proces – program w stadium wykonywania zawierający:
 - informacje dotyczące zbioru wykorzystywanych zasobów
 - dane kernela,
 - stan procesora,
 - przestrzeń adresową pamięci z jednym bądź większą ilością odwzorowań,
 - jeden bądź więcej wykonywanych wątków,
 - sekcję danych (zmienne globalne)



Wątek

- Obiekt aktywności procesu zawierający własny:
 - unikatowy wskaźnik instrukcji – informacje dotyczące aktualnie wykonywanej instrukcji bądź instrukcji wykonywanej jako kolejna po bieżącej,
 - stos,
 - zbiór rejestrów procesora,



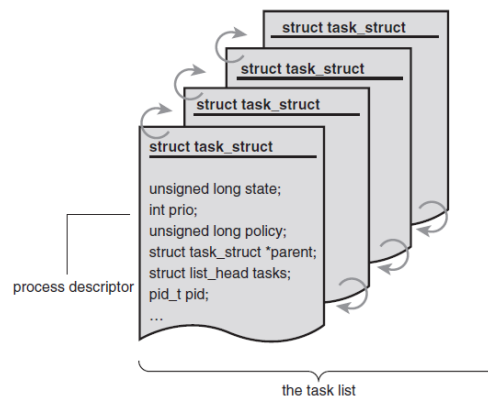
Proces

- dla systemu Linux wątek jest specjalnym typem procesu,
- Z procesem bezpośrednio związane są:
 - wirtualizacja procesora – stwarza procesowi iluzję, że ma on dostępny cały system
 - wirtualizacja pamięci – pozwala procesowi na alokację oraz zarządzanie pamięcią w sposób, jakby posiadał on całą pamięć systemową,
- proces tworzony jest przy pomocy instrukcji systemowej `fork()` lub `vfork()`,



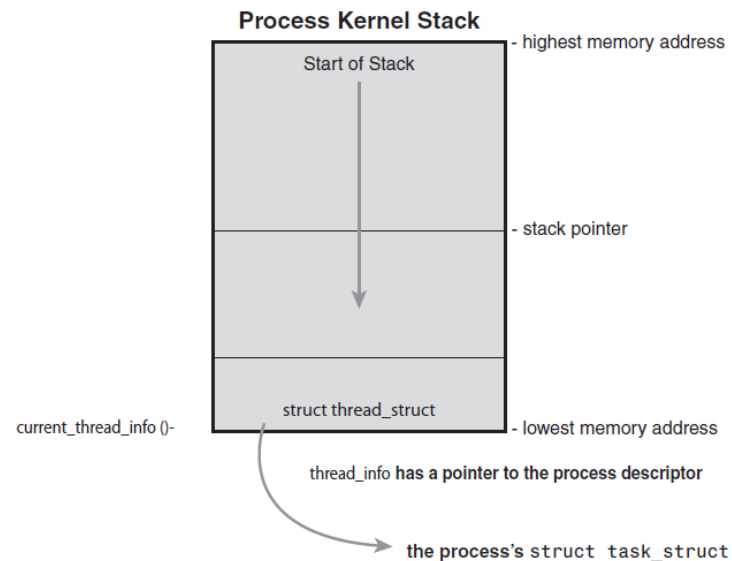
Deskryptor procesu

- Kernel zapamiętuje listę procesów w dwukierunkowej liście cyklicznej nazywanej listą zadań (*task list*), w której każdy element stanowi deskryptor procesu - `struct task_struct` (zdefiniowany w `<linux/sched.h>`)
- deskryptor procesu posiada wszystkie informacje dotyczące opisywanego procesu – 1.7KB na maszynie 32bitowej,
- alokowany jest poprzez *slab allocator* – mechanizm zarządzania pamięcią odpowiedzialny m.in. za efektywność alokacji oraz za niwelację fragmentacji pamięci,
- przed wersją 2.6 zapamiętywany na końcu stosu każdego procesu – optymalizacja dla architektur posiadających małą liczbę rejestrów (np. x86)



struct thread_info

- struktura (<asm/thread_info.h>) tworzona wraz z task_struct,
- zawiera adres struktury task_struct,
- alokowana na końcu stosu procesu opisywanego przez nią,



Identyfikacja procesu

- System identyfikuje proces po jego unikatowym identyfikatorze (PID) – w celu zachowania kompatybilności maksymalny rozmiar to 32 768 (short int) – rozmiar ten może być zwiększony (kontrolowane przez `<linux/threads.h>`) – zmiana poprzez `/proc/sys/kernel/pid_max`
- PID zachowany jest w każdym deskryptorze procesu
- *current* – makro wskazujące na strukturę *task_struct* aktualnie wykonywanego procesu opisanego przez *thread_info* (np. na x86 – *current* obliczane jest poprzez maskowanie 13 najmniej znaczących bitów wskaźnika stosu – `current_thread_info()`)



Stany procesu

- TASK_RUNNING
- TASK_INTERRUPTIBLE
- TASK_UNINTERRUPTIBLE
- __TASK_TRACED
- __TASK_STOPPED

Zmiany stanu procesu kernel wykonuje przy pomocy funkcji `set_task_state(task, state)`, bądź `set_current_state(state)`.



Organizacja procesów

- każdy proces jest potomkiem procesu *init* (PID 1), proces *init* uruchamiany jest w ostatnim krokiem uruchamiania systemu,
- deskryptor procesu *init* jest statycznie alokowany jako *init_task*,
- każdy proces ma dokładnie jednego rodzica,
- każdy proces ma zero lub więcej dzieci,
- procesy posiadające ten sam proces jako rodzica to procesy pokrewne (*siblings*),
- relacja pokrewieństwa zapamiętana jest w deskryptorze procesu (*task_struct*) – *parent*, *children*,



Tworzenie procesu

- `fork()` – tworzy proces dziecko będący kopią procesu wywołującego tę funkcję (różnica w PID, PPID, niektórych zasobach oraz statystykach, które nie są dziedziczone), implementowany poprzez funkcję systemową `clone()`, która wywołuje funkcję `do_fork()` odpowiedzialną za większość prac związanych z tworzeniem nowego procesu, `do_fork()` wywołuje m.in. funkcję `copy_process()`. Gdy `copy_process()` zakończyła się bezbłędnie – nowy proces jest budzony i uruchamiany.
- `exec()` – ładuje kod wykonywalny do przestrzeni adresowej i rozpoczyna jego działanie

W innych systemach *fork* i *exec* połączony jest w mechanizm *spawn*

COW - Technika odwlekająca bądź zapobiegająca kopiowaniu danych (każdej strony pamięci w przestrzeni adresowej danego procesu) w momencie tworzenia procesu.



fork() – do_fork() – copy_process() (1)

- `dup_task_struct()` – tworzy nowy stos kernela, `thread_info`, `task_struct` (deskryptor procesu dziecka jest identyczny z deskryptorem procesu rodzica),
- sprawdza czy nowy proces nie wyczerpie zasobów przeznaczonych dla bieżącego użytkownika,
- ustawia niektóre informacje w deskryptorze procesu dziecka na wartości domyślne, np. statystyki. Większość informacji zawartych w `task_struct` pozostaje jednak niezmieniona,
- stan nowego procesu ustawiany jest na `TASK_UNINTERRUPTIBLE` – zapewnia, że nie jest on jeszcze uruchomiony,
- ...



fork() – do_fork() – copy_process() (2)

- ...
- copy_flags() – aktualizacja pola flags dla procesu dziecka,
- PF_SUPERPRIV = 0, PF_FORNOEXEC=1,
- alloc_pid() – przypisanie nowego PID,
- W zależności od przekazanych flag do funkcji clone() –
duplikowane bądź współdzielone używane zasoby, informacje
dotyczące systemu plików, przestrzeni adresowej, przestrzeni
nazw oraz signal handlers,
- zwraca wskaźnik do nowopowstałego procesu dziecka



vfork()

- Ten sam efekt, co w przypadku fork() poza kopiowaniem od rodzica wpisów do tablicy stron,
- Utworzony proces wykonuje się w przestrzeni adresowej rodzica, blokując jego do czasu wywołania funkcji exec() bądź exit(),
- Gdy zostanie wprowadzony COW dla tablicy stron, vfork() przestanie być potrzebny.



Wątki

- ten sam program – współdzielona przestrzeń adresowa,
- kernel Linuxa implementuje wątki jako standardowe procesy (nie udostępnia specjalnej semantyki harmonogramowania bądź specjalnych struktur w celu reprezentacji wątku – wątek reprezentowany jest przez `task_struct`)
- wątek

`clone (CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND,0)`

- proces

`fork()` - `clone(SIGCHLD, 0)`

`vfork()` – `clone (CLONE_VFORK | CLONE_VM | SIGCHLD, 0)`



Kernel threads (1)

`ps -ef`

- Wątki te nie posiadają przestrzeni adresowej (wskaźnik *mm* jest ustawiony na NULL),
- operują tylko w przestrzeni kernela (nie przełączają kontekstu),
- Tak samo jak procesy kernela podlegają harmonogramowi oraz mogą zostać wywłaszczone,
- Wątek kernela może być tworzony tylko przez inny wątek kernela (*kthreadd* – kernel thread daemon – rodzic wszystkich wątków kernela)

```
struct task_struct *kthread_create(  
    int (*threadfn) (void *data),  
    void *data,  
    const char namefmt[],...)
```



Kernel threads (2)

- `wake_up_process()` – budzi proces,
- `kthread_run` – makro tworzące wątek oraz go uruchamiające
(`kthread_create()`+`wake_up_process()`)
- Zakończenie pracy wątku – samoczynnie - `do_exit()`
bądź arbitralnie - `kthread_stop(task_struct)`,



do_exit() (1)

- ustawia flagę PF_EXITING w polu flags w strukturze task_struct,
- del_timer_sync() – deaktywacja timera – w odróżnieniu od del_timer() – gwarantowane, że żaden nie jest zakolejkowany bądź uruchomiony nawet na innym procesorze,
- If BSD process accounting ins enabled, do_exit() calls acct_update_integrals() to write out accounting information,
- exit_mm() – zwolnienie mm_struct, gdy nie jest ona wykorzystywana przez inny proces (współdzielenie) – kernel usuwa ją,
- exit_sem() – gdy proces oczekuje na semafor IPC – jest usuwany z kolejki,
- ...



do_exit() (2)

- ...
- `exit_files()`, `exit_fs()` – dekrementacja licznika użycia obiektów skojarzonych z deskryptorami plików oraz danych dotyczących systemu plików (tak samo jak w przypadku `exit_mm` – gdy licznik jest 0 – obiekt jest usuwany),
- Ustawienie `exit_code` w `task_struct` na kod przekazany do instrukcji `exit()` bądź kod określający, że proces został przerwany przez kernela. Pole `exit_code` przeznaczone jest do opcjonalnego pobrania przez proces rodzica.
- `exit_notify()` – informowanie procesu rodzica o zakończeniu działania, przypisanie dzieciom nowego rodzica – `forget_original_parent()` – `find_new_reaper()` (procesu znajdującego się w tej samej grupie procesów bądź procesowi *init*), ustawienie `exit_state` w `task_struct` na `EXIT_ZOMBIE`,
- `schedule()` – przełączenie na inny proces – z uwagi, że proces nie jest już poddawany harmonogramowaniu jest to ostatnia funkcja przez niego wykonywana, `do_exit()` nigdy nic nie zwraca



Usuwanie deskryptora procesu (1)

- po wykonaniu funkcji `do_exit()` proces przechodzi do stadium `EXIT_ZOMBIE`, pozostaje zaalokowana jeszcze pamięć w postaci: stosu kernela, struktury `thread_info` i `task_struct`. Informacje te po odebraniu przez proces rodzica (`wait()`) bądź po oświadczeniu, że nie jest on nimi zainteresowany pamięć ta jest zwalniana i udostępniana systemowi do ponownego użycia.
- Znajdowanie procesów zombie `ps - el | grep 'Z'`



Usuwanie deskryptora procesu (2)

- W celu usunięcia deskryptora procesu wołana jest funkcja `release_task()`, która:
 - wywołuje `__exit_signal()` wywołującą `__unhash_process()`, która wywołuje `__detach_pid()` w celu usunięcia PID z tablicy hashującej oraz usuwa proces z listy,
 - `__exit_signal()` zwalnia wszystkie pozostałe zasoby używane przez teraz już martwy proces i finalizes statistics and bookkeeping,
 - gdy proces był ostatnim procesem z grupy procesów i proces główny jest procesem zombie, `release_task()` informuje rodzica procesu głównego z danej grupy,
 - `release_task()` – woła `put_task_struct()` w celu:
 - zwolnienia stron zawierających stos procesu kernela,
 - dealokacji `thread_info` oraz slab cache zawierającego `task_struct`.

